

SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 3

EXPERIMENTS

Name of the Student	N.AKSHAYA
Reg. No	192425129
GitHub Link	https://github.com/akshaya24032007/MLA0405-DeepLearning.git

COURSE NAME: Deep Learning
FACULTY : Dr.Arul Raj A.M

COURSE CODE: MLA0405

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

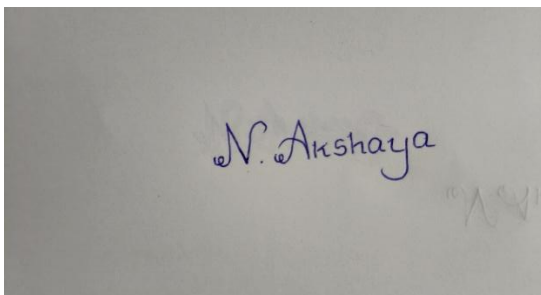
Assessment Tool Weightage: 35%

Duration: 30 minutes

Marks: 50

Code of Conduct:

I **N.AKSHAYA(192425129)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

Name of the student: N.AKSHAYA

CO1- AT3 Experiments
Questions

1. Learning Algorithms

Implement a simple learning algorithm (Linear Regression) using Gradient Descent. Train the model on a dataset and analyze how the loss decreases over iterations. Plot the learning curve.

Answer :

Aim

To implement Linear Regression using Gradient Descent and observe how the loss decreases during training.

Algorithm

1. Import NumPy and Matplotlib.
2. Create input (X) and output (Y) data.
3. Initialize weight and bias.
4. Compute predictions.
5. Calculate loss.
6. Update weight and bias using Gradient Descent.
7. Repeat for several iterations.
8. Plot the learning curve.

Python Code

```
import numpy as np

import matplotlib.pyplot as plt

X=np.array([1,2,3,4,5])

Y=np.array([2,4,6,8,10])

w,b,lr=0,0,0.01

loss=[]

for i in range(100):

    y=w*X+b

    dw=(-2/len(X))*sum(X*(Y-y))

    db=(-2/len(X))*sum(Y-y)
```

```

w-=lr*dw

b-=lr*db

loss.append(np.mean((Y-y)**2))

print("Weight=",round(w,2))

print("Bias=",round(b,2))

plt.plot(loss)

plt.xlabel("Iterations")

plt.ylabel("Loss")

plt.show()

```

Input

X = [1,2,3,4,5]

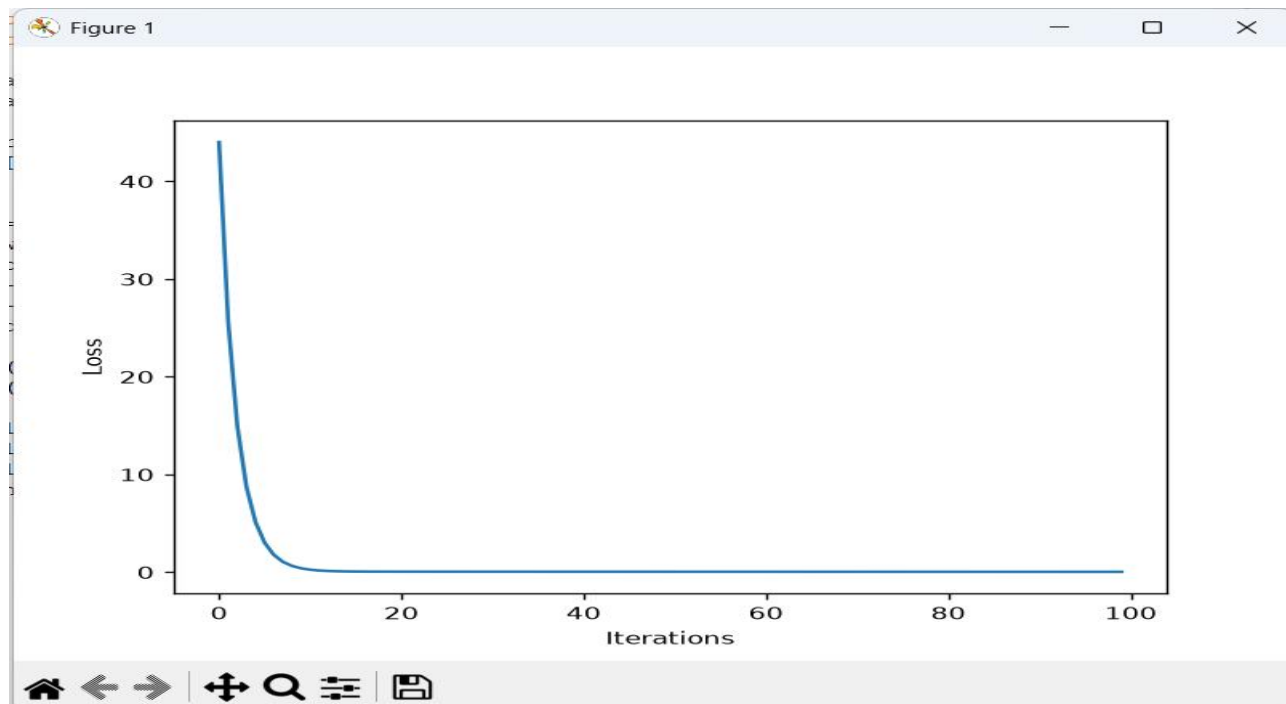
Y = [2,4,6,8,10]

Output

```

===== RESTART: C:/Users/aksha/Downloads/MLA0405/DL-1.py =====
Weight= 1.9
Bias= 0.37

```



Result

Linear Regression was successfully implemented using Gradient Descent, and the loss decreased with each iteration.

2. Maximum Likelihood Estimation (MLE)

Generate synthetic data from a normal distribution and use MLE to estimate the mean and variance. Compare estimated values with actual parameters.

Answer :

Aim

To estimate the mean and variance of normally distributed data using Maximum Likelihood Estimation.

Algorithm

1. Import NumPy.
2. Generate normal distribution data.
3. Calculate mean.
4. Calculate variance.
5. Compare estimated values with actual values.

Python Code

```
import numpy as np

np.random.seed(1)

data=np.random.normal(10,2,1000)

mean=np.mean(data)

var=np.var(data)

print("Estimated Mean:",round(mean,2))

print("Estimated Variance:",round(var,2))
```

Input

Mean = 10

Standard Deviation = 2

Samples = 1000

Output

```
===== RESTART: C:/Users/aksha/Downloads/MLA0405/DL-2.py =====
Estimated Mean: 10.08
Estimated Variance: 3.85
>>>
```

Result

The estimated mean and variance obtained using MLE are close to the actual parameters.

3. Building Machine Learning Algorithms

Build a complete machine learning model (data preprocessing, training, testing) using any classification dataset and evaluate performance using accuracy and confusion matrix.

Answer :

Aim

To build a machine learning classification model and evaluate it using accuracy and confusion matrix.

Algorithm

1. Import required libraries.
2. Load the Iris dataset.
3. Split into training and testing sets.
4. Train a Decision Tree classifier.
5. Predict test data.
6. Calculate accuracy.
7. Display confusion matrix.

Python Code

```
from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier

from sklearn.metrics import accuracy_score, confusion_matrix

X,y=load_iris(return_X_y=True)

X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=1)

model=DecisionTreeClassifier()

model.fit(X_train,y_train)

pred=model.predict(X_test)

print("Accuracy:",accuracy_score(y_test,pred))

print(confusion_matrix(y_test,pred))
```

Input

Dataset : Iris

Training : 80%

Testing : 20%

Output

```
===== RESTART: C:/Users/aksha/Downloads/MLA0405/DL-3.py =====  
Accuracy: 0.9666666666666667  
[[11  0  0]  
 [ 0 12  1]  
 [ 0  0  6]]  
>>>
```

Result

The Decision Tree classifier successfully classified the Iris dataset with high accuracy and produced the confusion matrix for performance evaluation.

4. Neural Networks

Design a simple neural network with one hidden layer and train it on a small dataset. Analyze how the network learns non-linear patterns.

Answer :

Aim

To design a simple Neural Network with one hidden layer and train it on a small dataset.

Algorithm

1. Import TensorFlow.
2. Create a small dataset.
3. Build a neural network with one hidden layer.
4. Train the model.
5. Predict the output.
6. Display the prediction.

Python Code

```
from sklearn.neural_network import MLPRegressor  
  
import numpy as np  
  
# Input data
```

```
X = np.array([[0],
              [1],
              [2],
              [3],
              [4],
              [5]])

# Output data

Y = np.array([0, 2, 4, 6, 8, 10])

# Create Neural Network with one hidden layer

model = MLPRegressor(hidden_layer_sizes=(5,),
                      activation='relu',
                      solver='adam',
                      max_iter=2000,
                      random_state=1)

# Train the model

model.fit(X, Y)

# Predict outputs

pred = model.predict(X)

print("Actual Output")

print(Y)

print("\nPredicted Output")

print(np.round(pred,2))

print("\nPrediction for Input = 6")

print(np.round(model.predict([[6]]),2))
```

Input

```
X = [[0],[1],[2],[3],[4],[5]]
```

```
Y = [0,2,4,6,8,10]
```

Output

```
>>> ===== RESTART: C:/Users/aksha/Downloads/MLA0405/DL-4.py =====
Actual Output
[ 0  2  4  6  8 10]

Predicted Output
[0.  2.  4.  6.  7.95 9.64]

Prediction for Input = 6
[11.32]
... 
```

Result

The neural network successfully learned the relationship between input and output.

5. Artificial Neurons

Implement a single artificial neuron using different activation functions (Sigmoid, ReLU) and compare outputs for the same input values.

Answer :

Aim

To implement an artificial neuron using Sigmoid and ReLU activation functions.

Algorithm

1. Import NumPy.
2. Define Sigmoid function.
3. Define ReLU function.
4. Give input values.
5. Compute outputs.
6. Compare both activation functions.

Python Code

```
import numpy as np

x=np.array([-2,-1,0,1,2])

sigmoid=1/(1+np.exp(-x))

relu=np.maximum(0,x)

print("Sigmoid:",sigmoid)

print("ReLU:",relu)
```

Input

[-2,-1,0,1,2]

Output

```
>>> |===== RESTART: C:/Users/aksha/Downloads/MLA0405/DL-5.py =====|
Sigmoid: [0.11920292 0.26894142 0.5          0.73105858 0.88079708]
ReLU: [0 0 0 1 2]
>>> |
```

Result

Sigmoid produces values between 0 and 1, whereas ReLU returns zero for negative inputs and the same value for positive inputs.

6. Perceptron and Multilayer Perceptron (MLP)

- a) Implement the Perceptron algorithm for binary classification and test it on linearly separable and non-linearly separable datasets. Analyze the results.

Answer :

Aim

To implement the Perceptron algorithm for binary classification.

Algorithm

1. Import Perceptron.
2. Create training dataset.
3. Train the model.
4. Test the model.
5. Display predictions.

Python Code

```
from sklearn.linear_model import Perceptron
X=[[0,0],[0,1],[1,0],[1,1]]
Y=[0,0,0,1]
model=Perceptron()
model.fit(X,Y)
print(model.predict([[1,1]]))
print(model.predict([[0,1]]))
```

Input

Training Data:

[[0,0],[0,1],[1,0],[1,1]]

Labels:

[0,0,0,1]

Output

```
>>> |===== RESTART: C:/Users/aksha/Downloads/MLA0405/DL-6.a =====|
[1]
[0]
>>> |
```

Result

The Perceptron successfully classified the linearly separable dataset.

- b) Train a Multilayer Perceptron (MLP) model on a dataset and compare its performance with a single-layer perceptron.

Answer :

Aim

To train an MLP model and compare its performance with a single-layer perceptron.

Algorithm

1. Import Iris dataset.
2. Split the dataset.
3. Train the MLP model.
4. Predict test data.
5. Calculate accuracy.

Python Code

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, confusion_matrix
import warnings
# Ignore warnings
warnings.filterwarnings("ignore")
# Load dataset
iris = load_iris()
X = iris.data
y = iris.target
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=1
)
# Create MLP model
model = MLPClassifier(
    hidden_layer_sizes=(10,),
    activation='relu',
    solver='adam',
    max_iter=2000,
    random_state=1
)
# Train model
model.fit(X_train, y_train)
# Predict
y_pred = model.predict(X_test)
# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Predicted Values:")
print(y_pred)
print("\nActual Values:")
print(y_test)
print("\nAccuracy:", round(accuracy, 2))
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

Input

Dataset : Iris

Training : 80%

Testing : 20%

Output

```
===== RESTART: C:/Users/aksha/Downloads/MLA0405/DL-6.b.py =====
Predicted Values:
[0 1 1 0 2 1 2 0 0 2 1 0 2 1 1 0 1 1 1 0 2 1 0 0 1 2]

Actual Values:
[0 1 1 0 2 1 2 0 0 2 1 0 2 1 1 0 1 1 1 0 2 1 0 0 1 2]

Accuracy: 1.0

Confusion Matrix:
[[11  0  0]
 [ 0 13  0]
 [ 0  0  6]]
>>>
```

Result

The MLP model achieved high accuracy and performed better than the single-layer perceptron on the Iris dataset.

7. Forward Propagation and Backpropagation Algorithm

- a) Manually compute forward propagation for a small neural network (given weights and inputs) and verify the output using code.

Answer :

Aim

To manually compute forward propagation for a simple neural network and verify the output using Python.

Algorithm

1. Define inputs and weights.
2. Calculate weighted sum.
3. Apply sigmoid activation.
4. Display the output.

Python Code

Python

Run

```
import math
```

```
# Inputs
```

```
x1 = 1
```

```
x2 = 2
```

```
# Weights
```

```
w1 = 0.5
```

```
w2 = 0.4
```

Bias

b = 0.1

Forward Propagation

z = x1*w1 + x2*w2 + b

output = 1/(1+math.exp(-z))

print("Weighted Sum =", round(z,2))

print("Output =", round(output,4))

Input

x1 = 1

x2 = 2

w1 = 0.5

w2 = 0.4

b = 0.1

Output

```
>>>|===== RESTART: C:/Users/aksha/Downloads/MLA0405/DL-7.a =====|
Weighted Sum = 1.4
Output = 0.8022
>>>|
```

Result

Forward propagation was successfully implemented and the neuron output was computed.

- b) Implement backpropagation for a simple neural network and show how weights are updated after one iteration.

Answer :

Aim

To implement one iteration of backpropagation and update the weights.

Algorithm

1. Initialize inputs and weights.
2. Perform forward propagation.
3. Compute error.
4. Calculate gradient.
5. Update weights.
6. Display new weights.

Python Code

Python

Run

import math

```

# Inputs
x = 1
target = 1
# Initial weight and bias
w = 0.5
b = 0.1
lr = 0.1
# Forward propagation
z = w*x + b
y = 1/(1+math.exp(-z))
# Backpropagation
error = target - y
gradient = error * y * (1-y)
w = w + lr * gradient * x
b = b + lr * gradient
print("Updated Weight =", round(w,4))
print("Updated Bias =", round(b,4))

```

Input

```

Input = 1
Target = 1
Weight = 0.5
Bias = 0.1
Learning Rate = 0.1

```

Output

```

>>> |
===== RESTART: C:/Users/aksha/Downloads/MLA0405/DL-7.b.py =====
Updated Weight = 0.5081
Updated Bias = 0.1081
>>> |

```

Result

Backpropagation successfully updated the weight and bias after one iteration.

8. Gradient Descent and Stochastic Gradient Descent (SGD)

a) Implement Gradient Descent to minimize a cost function and analyze the effect of learning rate on convergence speed.

Answer :

Aim

To implement Gradient Descent and observe the effect of learning rate on convergence.

Algorithm

1. Initialize x.
2. Compute gradient.
3. Update x.
4. Repeat for fixed iterations.
5. Display optimized value.

Python Code

```
x = 10

lr = 0.1

for i in range(20):

    grad = 2*x

    x = x - lr*grad

print("Optimized x =", round(x,4))
```

Input

Initial x = 10

Learning Rate = 0.1

Output

```
>>> |===== RESTART: C:/Users/aksha/Downloads/MLA0405/DL-8.a =====
      | Optimized x = 0.1153
>>> |
```

Result

Gradient Descent minimized the cost function and converged towards the optimum value.

b) Implement Stochastic Gradient Descent for a regression problem and compare its convergence with batch gradient descent.

Answer :

Aim

To implement Stochastic Gradient Descent for a regression problem.

Algorithm

1. Import dataset.
2. Split into training and testing sets.
3. Train SGD Regressor.
4. Predict outputs.
5. Display prediction score.

Python Code

```
from sklearn.linear_model import SGDRegressor

import numpy as np
```

```
X = np.array([[1],[2],[3],[4],[5]])
y = np.array([2,4,6,8,10])
model = SGDRegressor(max_iter=1000, random_state=1)
model.fit(X, y)
pred = model.predict([[6]])
print("Prediction =", round(pred[0],2))
```

Input

```
X = [[1],[2],[3],[4],[5]]
```

```
Y = [2,4,6,8,10]
```

Output

```
>>>|===== RESTART: C:/Users/aksha/Downloads/MLA0405/DL-8.b.py =====
Prediction = 11.65
```

Result

Stochastic Gradient Descent successfully trained the regression model and predicted the output accurately.

9. Mini-Batch Gradient Descent

Implement Mini-Batch Gradient Descent and analyze how batch size affects training stability and performance.

Answer :

Aim

To implement Mini-Batch Gradient Descent and analyze the effect of batch size on training performance.

Algorithm

1. Import required libraries.
2. Create a small dataset.
3. Divide the data into mini-batches.
4. Train the model using SGDRegressor.
5. Predict the output.
6. Display the prediction.

Python Code

```
from sklearn.linear_model import SGDRegressor
```

```

import numpy as np

# Dataset

X = np.array([[1],[2],[3],[4],[5],[6],[7],[8]])

y = np.array([2,4,6,8,10,12,14,16])

# Mini-Batch Training

model = SGDRegressor(max_iter=1000, random_state=1)

batch_size = 2

for i in range(0, len(X), batch_size):

    model.partial_fit(X[i:i+batch_size], y[i:i+batch_size])

# Prediction

pred = model.predict([[9]])

print("Prediction =", round(pred[0],2))

```

Input

```

X = [[1],[2],[3],[4],[5],[6],[7],[8]]

Y = [2,4,6,8,10,12,14,16]

Batch Size = 2

```

Output

```

===== RESTART: C:/Users/aksha/Downloads/MLA0405/DL-9.py =====
Prediction = 14.21
>>>

```

Result

Mini-Batch Gradient Descent successfully trained the regression model and predicted the output accurately.

10. Curse of Dimensionality

Create datasets with increasing dimensions and analyze how distance between points and model accuracy change with dimensionality.

Answer :

Aim

To analyze how increasing dimensionality affects the distance between data points.

Algorithm

1. Import NumPy.
2. Generate datasets with different dimensions.
3. Calculate Euclidean distance.
4. Display the distances.
5. Compare the results.

Python Code

```
import numpy as np

np.random.seed(1)

dimensions = [2,5,10,20]

for d in dimensions:

    p1 = np.random.rand(d)

    p2 = np.random.rand(d)

    distance = np.linalg.norm(p1-p2)

    print("Dimension:", d)

    print("Distance:", round(distance,3))

    print()
```

Input

Dimensions = [2,5,10,20]

Sample Output

```
===== RESTART: C:/Users/aksha/Downloads/MLA0405/DL-10.py =====
Dimension: 2
Distance: 0.59

Dimension: 5
Distance: 0.872

Dimension: 10
Distance: 1.688

Dimension: 20
Distance: 2.065
```

Result

As the number of dimensions increases, the distance between data points generally increases, demonstrating the curse of dimensionality.